

## Lab 5

### Part A - Scope

1. Create a global variable `course_name` and a function that creates a local variable with the same name. Print both and explain the result.
2. Create a function with a local counter and show that it does not remain available outside the function.
3. Create a function that attempts to modify a global numeric variable without `global`. Observe/describe the problem, then rewrite the design to return the new value instead.
4. Create a nested function and demonstrate a simple enclosing-scope lookup.
5. Create examples that avoid shadowing built-ins such as `list`, `str`, `sum` and `max`.

### Part B - \*args

1. Write `add_all(*numbers)` returning the sum without `sum()`.
2. Write `average(*numbers)`. Decide what should happen when no numbers are supplied.
3. Write `longest_word(*words)` returning the longest word.
4. Write `build_sentence(separator, *words)` returning one joined string.
5. Write `describe_scores(student_name, *scores)` returning name, number of scores and average.

### Part C - Positional unpacking

1. Create a list `[10, 20, 30]` and unpack it into a function expecting three positional parameters.
2. Create a tuple containing `first_name`, `last_name`, `city` and call a function using `*tuple`.
3. Use starred assignment: `first, *middle, last = values`. Test with several list lengths.
4. Explain in comments the difference between `*` in a function definition and `*` in a function call.

## Part D - **\*\*kwargs**

1. Write `show_profile(**info)` and iterate over all key/value pairs.
2. Write `create_user(username, **details)` returning one dictionary containing username plus all supplied details.
3. Write `build_product(name, price, **metadata)` returning a dictionary.
4. Write a function that accepts `**settings` and returns only settings whose value is not `None`.
5. Call a normal named-parameter function using `**dictionary unpacking`. Ensure dictionary keys match parameter names.

## Part E - **Combining parameters**

1. Create `log_event(event_type, *messages, **metadata)` returning a structured dictionary.
2. Create `calculate_order(customer, *prices, **options)`. Support an optional discount and shipping fee in options.
3. Create a function where explicit named parameters would be clearer than `**kwargs`. Write both versions and compare readability in comments.
4. Create at least three calls to the same flexible function with substantially different numbers of arguments.

## Part F - Applied challenge: Report builder

1. Build a flexible report system without files. `create_report(title, *sections, **metadata)` should return a dictionary.
2. Each section can be a string or a small dictionary; choose and document your design.
3. Metadata may include author, department, version, confidential and date.
4. Write `summarize_report(report)` that returns a readable multi-line string.
5. Write `count_words(*sections)` that counts words across all supplied textual sections.
6. Use dictionary unpacking to create at least two reports from predefined metadata dictionaries.
7. Demonstrate at least one case where your function deliberately ignores or handles a missing optional metadata field.

## Part G - Stretch challenges

1. Write `merge_settings(defaults, **overrides)` returning a new dictionary without modifying defaults.
2. Write `call_summary(function_name, *args, **kwargs)` returning a string describing what would be called.
3. Write a flexible statistics function that returns count, total, average, min and max for `*numbers`. Implement the calculations manually where reasonable.
4. Create five 'predict the output' scope questions and verify your predictions.